

第 2 讲

Introduction to CUDA C and Data Parallel Programming

目录

1 核心内容	1
2 从线程到大规模数据并行	1
2.1 线程是计算的基本单位	1
2.2 “多个线程”与“海量线程”	1
2.3 任务并行与数据并行	2
3 CUDA 的主机-设备执行模型	3
3.1 一个源程序，两类执行位置	3
3.2 核函数与单程序多数据模型	4
4 线程层次结构与一维索引映射	5
4.1 网格、线程块与线程	5
4.2 全局线程索引的推导	6
4.3 线程索引是“数据归属规则”	7
5 向量加法：从顺序循环到线程网格	7
5.1 顺序基线与并行分解	8
5.2 CPU 与 GPU 的分离内存空间	8
5.3 设备内存管理 API	10
6 核函数实现、启动配置与边界检查	11
6.1 核函数声明与线程工作	11
6.2 三重尖括号与启动参数	12
6.3 向上取整除法的推导	12
6.4 为什么必须做边界检查	13
6.5 向量加法的主机端骨架	14
7 函数限定符与 CUDA 编译流程	14
7.1 三类函数限定符	14
7.2 从.cu 源文件到 CPU/GPU 机器代码	15
8 异步执行与错误检查	16
8.1 核函数启动默认是异步的	16
8.2 两类错误：启动错误与执行错误	17
9 完整、可检查的向量加法程序	18
9.1 程序结构	20
10 例题与完整推导	21
10.1 例题一：读取核函数启动配置	21
10.2 例题二：每线程处理 8 个元素	21
10.3 例题三：局部变量有多少份	22
10.4 特殊长度输入	22
11 常见错误、检查清单与公式表	22

11.1 常见错误	22
11.2 写一维 CUDA 核函数前的检查清单	23
11.3 核心公式表	24
11.4 术语速查	24
12 综合框架与关键结论	25

1 核心内容

内容概览

- 数据并行计算 (*data-parallel computing*) 的基本概念;
- **CUDA C 编程接口** (*CUDA C programming interface*) 的基本特性。

2 从线程到大规模数据并行

2.1 线程是计算的基本单位

线程 (*thread*) 是一条正在执行的顺序计算。线程包含程序或指令序列、指出当前执行位置的 **程序计数器** (*program counter, PC*), 以及执行上下文 (*context*), 例如内存状态与寄存器状态。单个线程内部仍按顺序执行; 并行性来自许多线程在硬件上同时推进。

线程的最小心智模型

设线程 T 的状态写成

$$T = (P, PC, R, M_{\text{visible}}),$$

其中 P 是程序, PC 是下一条待执行指令的位置, R 表示该线程可见的寄存器状态, M_{visible} 表示该线程可以访问的内存状态。**CUDA** 中同一**核函数** (*kernel*) 产生的线程执行相同程序 P , 但它们看到的线程索引不同, 因此访问的数据位置不同。

2.2 “多个线程”与“海量线程”

传统多核 **CPU** 往往运行数量相对有限、行为较复杂的线程; **GPU** 的设计目标则是让数量极大的线程并发执行。当线程数量达到成千上万甚至更多时, 任意而频繁的线程间通信会迅速造成等待与串行化。因此, 最容易映射到 **GPU** 的工作负载通常具备以下特征:

- 线程之间的依赖很少;
- 每个线程能够独立确定自己的输入与输出;
- 大量线程执行相同或相近的计算;
- 线程之间即使合作, 也尽量限制在较小的局部范围内, 例如同一线程块。

CUDA 线程仍可通过共享内存(*shared memory*)、原子操作(*atomic operation*)与屏障同步(*barrier synchronization*)进行合作。设计并行程序时,应先寻找独立工作,再加入必要合作。

2.3 任务并行与数据并行

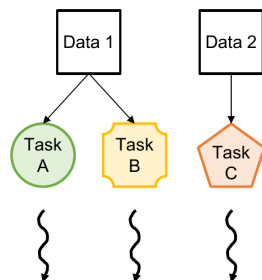
任务并行(*task parallelism*)把不同操作分配给不同执行单元;数据并行(*data parallelism*)则让大量执行单元在不同数据上执行相同操作。GPU 最擅长后者,因为大型数据集常能释放出极高的并行度。

Types of Parallelism



Task Parallelism

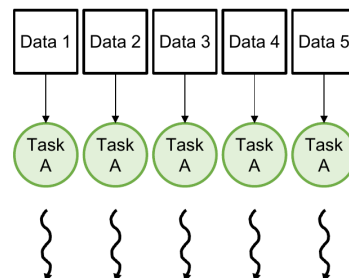
- Different operations performed on same or different data
- Usually, a modest number of tasks unleashing a modest amount of parallelism



ELECTRICAL & COMPUTER ENGINEERING

Data Parallelism

- Same operations performed on different data
- Potentially massive amounts of data unleashing massive amounts of parallelism
 - Most suitable for GPUs



GRAINGER ENGINEERING

图 1 任务并行 (task parallelism) 与数据并行 (data parallelism) 的对比

可用一个统一的数学形式描述数据并行。设输入数据为

$$X = \{x_0, x_1, \dots, x_{n-1}\},$$

每个输出元素由同一个函数 f 产生:

$$y_i = f(x_i), \quad 0 \leq i < n.$$

若 y_i 的计算不依赖尚未生成的 y_j , 也不修改其他线程负责的输出位置, 则所有 y_i 可以并行计算。

2.3.1 彩色图像转灰度图像

每个像素具有红、绿、蓝三个分量 (r_i, g_i, b_i) 。常用的亮度近似为

$$L_i = 0.299r_i + 0.587g_i + 0.114b_i.$$

对任意两个不同像素 $i \neq j$, L_i 的计算不读取或修改像素 j 的输出, 因此像素转换彼此独立。
顺序循环

for each pixel: $L_i \leftarrow f(r_i, g_i, b_i)$

可以被解释成“一个线程对应一次循环迭代”。这正是 CUDA 入门中最重要的循环并行化 (*loop parallelism*) 视角。

判断数据并行时必须问的三个问题

1. 每次迭代写入的位置是否唯一?
2. 每次迭代读取的数据是否已经可用, 而非由其他并行迭代临时产生?
3. 改变不同迭代的执行顺序, 是否仍能得到相同结果?

三个答案均为“是”时, 通常可以直接采用一线程一元素的映射。

3 CUDA 的主机-设备执行模型

3.1 一个源程序, 两类执行位置

CUDA 程序是异构应用 (*heterogeneous application*): 同一个工程同时包含在 CPU 上运行的主机代码 (*host code*), 以及在 GPU 上运行的设备代码 (*device code*)。主机代码通常负责串行或并行度较低的工作, 例如输入输出、内存管理与核函数调度; 设备代码负责数据量大、并行度高的计算阶段。

CUDA/OpenCL Execution Model



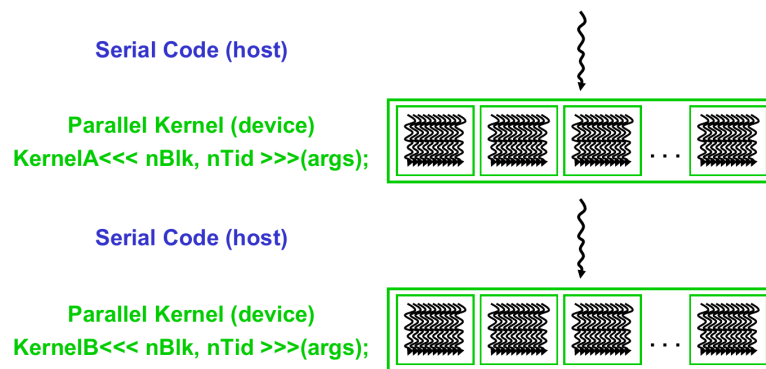
• Integrated Host + Device Application (C Program)

• Host Code (C):

- Handles serial or modestly parallel tasks

• Device Kernel (C, SPMD Model):

- Executes highly parallel sections of the program



ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 2 CUDA/OpenCL 的集成式主机-设备执行模型

一个典型的时间顺序可以写成：

主机准备数据 → 启动核函数 A → 主机继续执行或等待 → 启动核函数 B → 取回结果。

3.2 核函数与单程序多数据模型

核函数 (kernel) 是由许多 GPU 线程执行的设备函数。一次核函数启动 (kernel launch) 创建一个线程网格。网格中的所有线程运行同一份核函数代码，但每个线程使用自己的索引选择不同数据。这种编程方式称为**单程序多数据 (Single Program Multiple Data, SPMD)**。

SPMD 的“同一程序”并不意味着所有线程在每一时刻必然完成相同工作。线程可以依据索引进行条件判断；只是它们的代码文本相同。最简单的向量加法核函数逻辑是

$$C[i] = A[i] + B[i],$$

其中线程索引 i 决定该线程访问哪一对输入元素。

主机与设备的职责边界

对向量加法而言，主机代码负责准备 A 、 B 、 C ，把输入移动到设备可访问的位置，配置线程组织并启动核函数；设备线程只负责执行元素级加法。把数据移动、调度和计算区分开，是阅读任何 CUDA 程序的第一步。

4 线程层次结构与一维索引映射

4.1 网格、线程块与线程

一次核函数启动创建一个**网格 (grid)**。网格由若干**线程块 (thread block)** 组成，每个线程块再由若干**线程 (thread)** 组成。对于一维网格：

- `gridDim.x`: 网格在 x 方向包含的线程块数；
- `blockIdx.x`: 当前线程块在网格中的索引；
- `blockDim.x`: 每个线程块在 x 方向包含的线程数；
- `threadIdx.x`: 当前线程在线程块内部的索引。

Thread Blocks: Scalable Cooperation



- Threads within a block cooperate via **shared memory, atomic operations** and **barrier synchronization** (to be covered later)
- Threads in different blocks cooperate less.

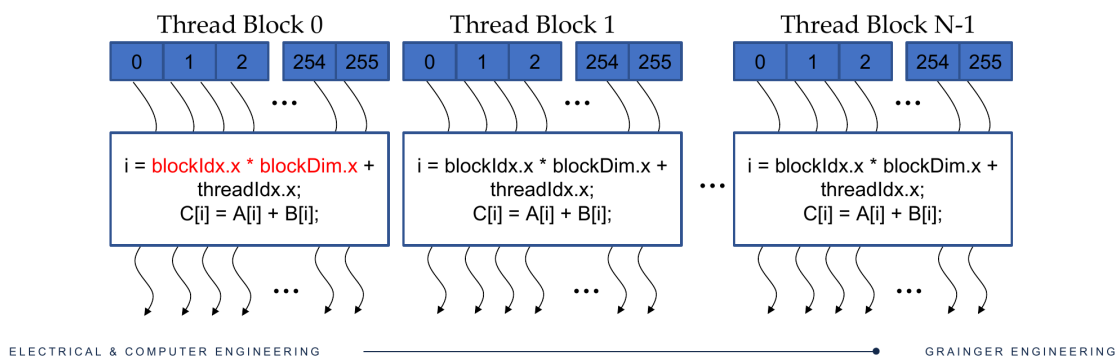


图 3 线程块内部可进行较强合作，不同线程块之间的合作更受限制

同一线程块内的线程可通过共享内存、原子操作与屏障同步进行局部合作。不同线程块必须能够以任意顺序调度，因此各线程块通常应设计为彼此独立。这个限制换来了可扩展性(*scalability*)：同一核函数可以在执行资源不同的 GPU 上运行，而无需把线程块绑定到特定硬件位置。

4.2 全局线程索引的推导

设每个线程块有 $T = \text{blockDim.x}$ 个线程，当前线程块索引为

$$b = \text{blockIdx.x},$$

线程在线程块内的局部索引为

$$t = \text{threadIdx.x}.$$

在当前线程块之前共有 b 个完整线程块，每个线程块贡献 T 个线程，所以当前线程块之前一共有 bT 个线程。把局部偏移 t 加上，得到全局一维索引

$$i = bT + t$$

即

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

4.2.1 覆盖范围证明

对固定的线程块 b ，局部索引满足 $0 \leq t < T$ ，因此该线程块产生的全局索引范围是

$$bT \leq i \leq bT + T - 1.$$

相邻线程块 $b + 1$ 的起始索引为

$$(b + 1)T = bT + T,$$

恰好接在前一线程块的末尾之后。因此所有线程块共同形成连续、不重叠的一维索引区间。

例：线程块大小为 256

若 $T = 256$ ：

线程块0： $i = 0, 1, \dots, 255$,

线程块1： $i = 256, 257, \dots, 511$,

线程块3中的线程0： $i = 3 \times 256 + 0 = 768$.

由此得到的全局索引与图示中的线程分配一致。

4.3 线程索引是“数据归属规则”

索引公式本身只是编号。真正的并行设计问题是：编号为 i 的线程应该负责哪一块数据？对一维向量加法，最自然的规则是线程 i 负责元素 i 。对二维图像、卷积、归约或扫描，线程到数据的映射会更复杂，但方法相同：先定义工作单元，再用内置索引计算该工作单元的位置。

关键结论

给定 B 个线程块、每块 T 个线程：

$$\text{启动线程总数} = BT.$$

每个线程拥有自己的自动局部变量副本；因此若核函数中声明一个局部变量 i ，整个网格生命周期中最多会为 BT 个线程各产生一个逻辑副本。

5 向量加法：从顺序循环到线程网格

5.1 顺序基线与并行分解

向量加法 (vector addition) 是数据并行程序中的 “Hello, World!”。给定长度均为 n 的向量 A 与 B ，目标是计算

$$C[i] = A[i] + B[i], \quad 0 \leq i < n.$$

顺序 C++ 实现用一个循环依次完成 n 次加法：

```
1 void vecAddCPU(const float* A, const float* B, float* C, int n) {  
2     for (int i = 0; i < n; ++i) {  
3         C[i] = A[i] + B[i];  
4     }  
5 }
```

并行化时，不需要保留这个循环的顺序执行结构。把第 i 次循环迭代定义为一个独立工作单元，再把工作单元 i 分配给全局线程 i ：

$$\text{线程 } i \longleftrightarrow C[i] = A[i] + B[i].$$

因此，原来的循环被整个线程网格替代。并行化前应先回答 “每个线程究竟独立负责什么工作”，因为这一选择会同时决定线程数量、数据布局与后续优化空间。

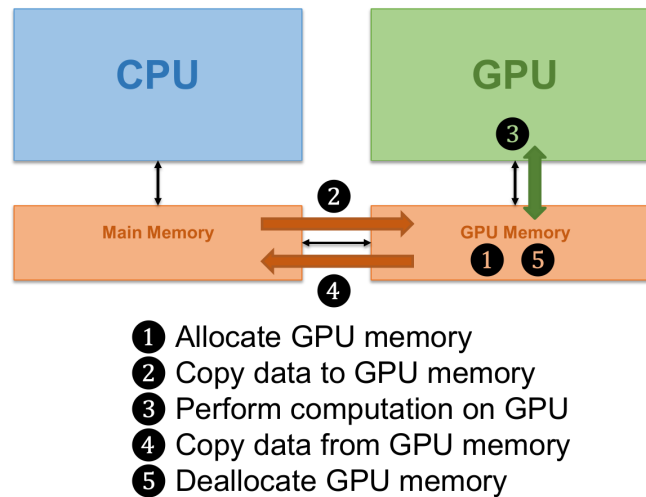
唯一写入者映射

输出元素 $C[i]$ 由唯一线程 i 负责写入。不同线程不会同时修改同一输出位置，因此不需要原子操作或同步。

5.2 CPU 与 GPU 的分离内存空间

经典的离散 GPU 系统模型由彼此分离的主机与设备内存空间组成：CPU 连接主机内存 (*host memory*)，GPU 连接设备全局内存 (*device global memory*)。主机指针默认指向主机地址空间，设备指针指向设备地址空间；在基础 CUDA 编程模型中，二者不能被对方代码直接解引用。

Memory Management



ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 4 设备内存管理的五个阶段

完整数据流由五步构成：

1. 在设备端分配空间；
2. 把输入数据从主机复制到设备；
3. 在设备端启动核函数并计算；
4. 把结果从设备复制回主机；
5. 释放设备端空间。

即使系统提供**统一内存 (unified memory)** 或某些物理共享内存机制，数据仍需要在合适时机出现在 GPU 可高效访问的位置。显式分配与显式复制能够清楚呈现数据移动的成本与依赖关系。

5.3 设备内存管理 API

5.3.1 分配与释放

CUDA API	作用与关键参数
<code>cudaMalloc(void **devPtr, size_t size)</code>	在设备全局内存中分配 <code>size</code> 字节，并把起始地址写入 <code>*devPtr</code> 。返回值类型为 <code>cudaError_t</code> 。
<code>cudaFree(void *devPtr)</code>	释放由 <code>devPtr</code> 指向的设备内存区域。

对 n 个 `float` 元素，字节数应写成

$$S = n \cdot \text{sizeof}(\text{float}).$$

实际代码宜用 `size_t` 保存 S ，避免大规模数组下的整数溢出。

```
size_t bytes = static_cast<size_t>(n) * sizeof(float);
float *A_d = nullptr, *B_d = nullptr, *C_d = nullptr;

cudaMalloc(reinterpret_cast<void**>(&A_d), bytes);
cudaMalloc(reinterpret_cast<void**>(&B_d), bytes);
cudaMalloc(reinterpret_cast<void**>(&C_d), bytes);
// ... use the allocations ...
cudaFree(A_d);
cudaFree(B_d);
cudaFree(C_d);
```

设备指针的语义

`A_d` 的数值是设备地址。主机代码可以把它传给 CUDA 运行时 API 或核函数，但不应在普通主机表达式中写 `A_d[i]`。设备端数据访问应发生在核函数中。

5.3.2 主机-设备数据复制

`cudaMemcpy` 的接口为

```
cudaError_t cudaMemcpy(void* dst, const void* src,
                       size_t count, cudaMemcpyKind kind);
```

第四个参数明确复制方向：

枚举常量	方向
<code>cudaMemcpyHostToHost</code>	主机到主机 (host to host)
<code>cudaMemcpyHostToDevice</code>	主机到设备 (host to device)
<code>cudaMemcpyDeviceToHost</code>	设备到主机 (device to host)
<code>cudaMemcpyDeviceToDevice</code>	设备到设备 (device to device)

向量加法的输入与输出复制分别为

```
cudaMemcpy(A_d, A_h, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(B_d, B_h, bytes, cudaMemcpyHostToDevice);
// kernel launch
cudaMemcpy(C_h, C_d, bytes, cudaMemcpyDeviceToHost);
```

读代码时可先检查 `dst` 与 `src` 是否和复制方向一致。例如, `cudaMemcpyDeviceToHost` 要求目标指针位于主机端、源指针位于设备端。

6 核函数实现、启动配置与边界检查

6.1 核函数声明与线程工作

核函数使用 `__global__` 限定符声明。每个线程执行同一函数体, 并根据内置索引计算自己的元素编号:

```
1 __global__ void vecAddKernel(const float* A_d,
2                             const float* B_d,
3                             float* C_d,
4                             int n) {
5     int i = blockIdx.x * blockDim.x + threadIdx.x;
6     if (i < n) {
7         C_d[i] = A_d[i] + B_d[i];
8     }
9 }
```

`i` 是自动局部变量 (*automatic local variable*)。逻辑上, 每个线程拥有自己的 `i`, 一个线程对它的赋值不会改变其他线程的副本。

6.2 三重尖括号与启动参数

主机通过三重尖括号语法 (*triple-chevron syntax*) 启动核函数：

```
vecAddKernel<<<numBlocks, threadsPerBlock>>>(A_d, B_d, C_d, n);
```

第一个执行配置参数是网格中的线程块数，第二个参数是每个线程块的线程数。二者的位置不能互换。

6.3 向上取整除法的推导

设数据元素数为 n ，每个线程块有 T 个线程。若每个线程只处理一个元素，需要的线程块数量为

$$B = \left\lceil \frac{n}{T} \right\rceil.$$

在整数算术中常写为

$$B = \left\lfloor \frac{n + T - 1}{T} \right\rfloor,$$

对应代码

```
int numBlocks = (n + threadsPerBlock - 1) / threadsPerBlock;
```

证明如下。令 $n = qT + r$ ，其中 $0 \leq r < T$ 。

- 若 $r = 0$ ，则 n 可被 T 整除，公式得到 $B = q$ ；
- 若 $r > 0$ ，则至少还需要一个线程块，公式得到 $B = q + 1$ 。

因此该整数表达式与向上取整完全等价。

Thread Assignment for vecAdd ($n = 1000$, block size = 256) I

```
vecAddKernel<<<ceil(n/256.0), 256>>>(...)
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) C[i] = A[i] + B[i];
```

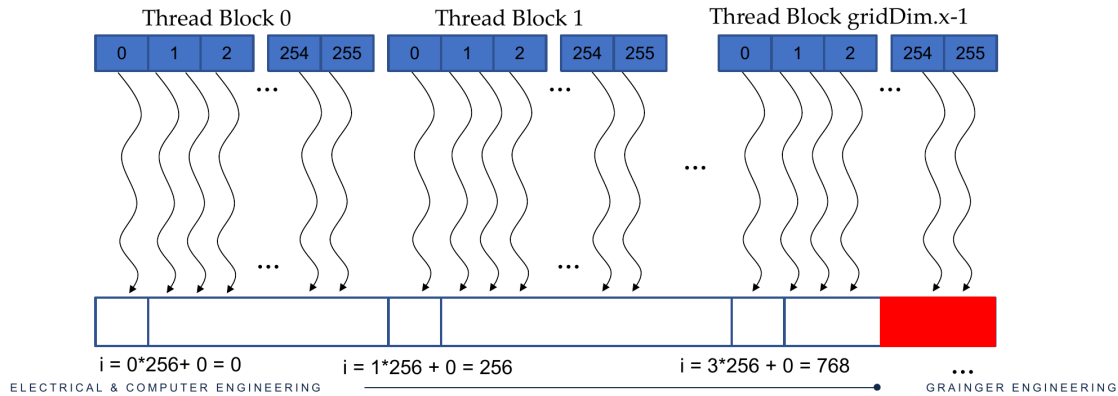


图 5 $n = 1000$ 、线程块大小为 256 时的线程分配

6.4 为什么必须做边界检查

总启动线程数为

$$N_{\text{launch}} = BT.$$

由于 B 经过向上取整，通常有 $N_{\text{launch}} \geq n$ 。多出的线程数

$$E = BT - n$$

满足 $0 \leq E < T$ 。这些额外线程没有对应的数据元素，必须在访问数组前检查

```
if (i < n) { /* valid work */ }
```

否则它们可能越界读取或写入设备内存。

例： $n = 1000$, $T = 256$

$$B = \left\lceil \frac{1000}{256} \right\rceil = 4, \quad N_{\text{launch}} = 4 \times 256 = 1024.$$

前 1000 个线程处理有效元素，最后 24 个线程满足 $i \geq 1000$ ，被边界条件屏蔽。

6.5 向量加法的主机端骨架

把内存管理与核函数启动串联起来，得到如下流程：

```
1 void vecAdd(const float* A_h, const float* B_h,  
2           float* C_h, int n) {  
3     size_t bytes = static_cast<size_t>(n) * sizeof(float);  
4     float *A_d = nullptr, *B_d = nullptr, *C_d = nullptr;  
5  
6     cudaMalloc(reinterpret_cast<void**>(&A_d), bytes);  
7     cudaMalloc(reinterpret_cast<void**>(&B_d), bytes);  
8     cudaMalloc(reinterpret_cast<void**>(&C_d), bytes);  
9  
10    cudaMemcpy(A_d, A_h, bytes, cudaMemcpyHostToDevice);  
11    cudaMemcpy(B_d, B_h, bytes, cudaMemcpyHostToDevice);  
12  
13    const int threadsPerBlock = 256;  
14    const int numBlocks = (n + threadsPerBlock - 1)  
15                        / threadsPerBlock;  
16    vecAddKernel<<<numBlocks, threadsPerBlock>>>(A_d, B_d, C_d, n);  
17  
18    cudaMemcpy(C_h, C_d, bytes, cudaMemcpyDeviceToHost);  
19  
20    cudaFree(A_d);  
21    cudaFree(B_d);  
22    cudaFree(C_d);  
23 }
```

这段代码展示了结构，但尚未加入系统化的错误检查。下一轮将补齐可运行版本，并解释异步启动、编译流程与错误传播。

7 函数限定符与 CUDA 编译流程

7.1 三类函数限定符

CUDA C++ 用函数限定符 (*function qualifier*) 说明函数从哪里调用、在哪里执行。

限定符	可调用位置	执行位置	含义
<code>__host__</code>	主机 (host)	主机 (host)	普通主机函数；省略限定符时的默认情况。
<code>__global__</code>	主机，或受支持的设备端启动	设备 (device)	核函数；调用时创建新的线程网格，返回类型必须为 <code>void</code> 。
<code>__device__</code>	设备 (device)	设备 (device)	由核函数或其他设备函数调用，不创建新网格。

同一个普通函数也可以同时标记为

```
__host__ __device__ float add(float a, float b) {
    return a + b;
}
```

编译器会为主机和设备分别生成版本。调用端仍需满足各自执行环境的限制。

7.2 从.cu 源文件到 CPU/GPU 机器代码

CUDA 源文件通常使用 `.cu` 扩展名，并交给 **NVIDIA CUDA 编译器** (*NVIDIA CUDA Compiler, NVCC*)。NVCC 根据函数限定符区分主机代码与设备代码：

1. 主机部分交给常规 C/C++ 编译器，生成 CPU 目标代码；
2. 设备部分先生成并行线程执行中间表示 (*Parallel Thread Execution, PTX*)；
3. PTX 在安装或运行阶段由即时编译器 (*just-in-time compiler, JIT compiler*) 转化为目标 GPU 的设备机器码，常以 SASS 表示。

Compiling A CUDA Program

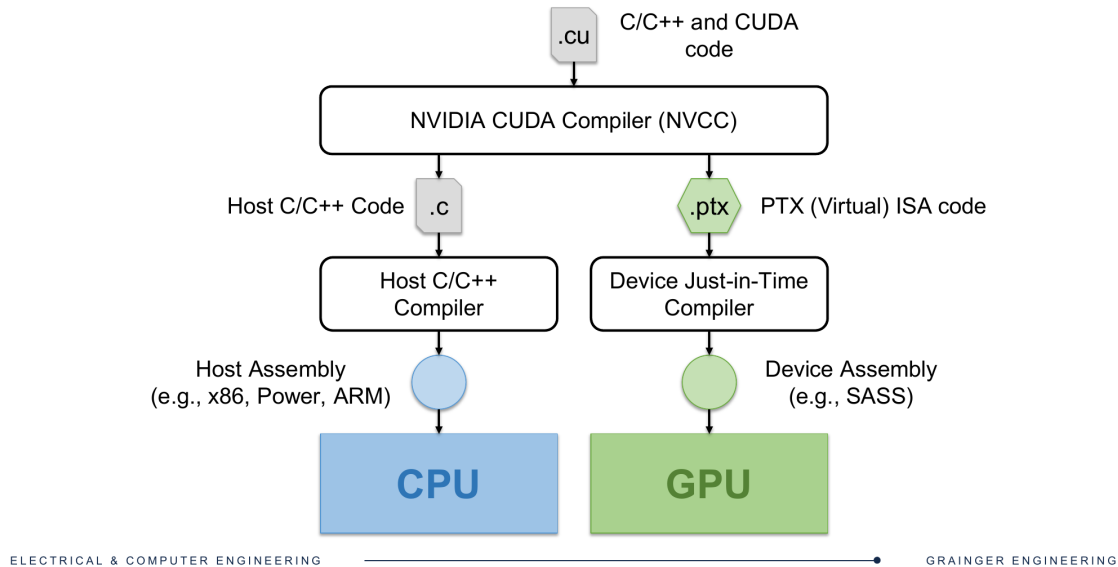


图 6 CUDA 程序的主机代码与设备代码编译路径

一个典型编译命令为

```
nvcc -O2 -std=c++17 vec_add.cu -o vec_add
```

其中 `-O2` 请求优化，`-std=c++17` 选择 C++ 语言标准。构建脚本或作业框架可以进一步封装这一步。

8 异步执行与错误检查

8.1 核函数启动默认是异步的

一次核函数启动通常先把工作提交给 GPU，然后让主机线程继续执行。此行为称为**异步调用** (*asynchronous call*)。因此，下面的 CPU 计时方式主要测到“提交请求”的时间，不一定覆盖 GPU 完成计算所需的时间：

```
start_timer();
vecAddKernel<<<B, T>>>(A_d, B_d, C_d, n);
stop_timer();           // usually too early
```

要让主机等待此前提交的设备工作完成，可调用

```
cudaDeviceSynchronize();
```

该函数提供主机与设备之间的**同步点** (*synchronization point*)，并返回此前异步工作中暴

露的错误状态。

Asynchronous Kernel Calls



- By default, kernel calls are asynchronous
 - Useful for overlapping GPU computations with CPU computations
- Use the following API function to wait for the kernel to finish

```
cudaError_t cudaDeviceSynchronize()
```

 - Blocks until the device has completed all preceding requested tasks
- Better ways of timing kernel execution:
 - Use CUDA events and stream synchronization
 - Use a profiler
 - We will cover these concepts later

ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 7 异步核函数调用与 `cudaDeviceSynchronize()`

当核函数后紧跟设备到主机的 `cudaMemcpy` 时，复制不能在产生结果的核函数完成之前得到最终输出。更精确的性能测量通常使用 **CUDA 事件(event)**，并在需要时配合**流同步(stream synchronization)** 或**性能分析器(profiler)**。

8.2 两类错误：启动错误与执行错误

绝大多数 CUDA 运行时 API 都返回 `cudaError_t`。`cudaSuccess` 表示成功。其他错误码可交给 `cudaGetErrorString`，转换为可读消息。

核函数调用本身没有普通返回值，因此常分两步检查：

1. `cudaGetLastError()`：检查非法启动配置等立即可见的**启动错误(launch error)**；
2. `cudaDeviceSynchronize()`：等待执行完成，并暴露越界访问等**异步执行错误(asynchronous execution error)**。

```
vecAddKernel<<<numBlocks, threadsPerBlock>>>(A_d, B_d, C_d, n);  
CUDA_CHECK(cudaGetLastError());  
CUDA_CHECK(cudaDeviceSynchronize());
```

简化示例与工程代码的区别

用于突出概念的短代码通常会省略错误检查，也可能包含简化写法。工程代码应检查内存分配、数据复制、核函数启动与核函数执行的返回状态。

9 完整、可检查的向量加法程序

以下完整示例把线程索引、内存管理、核函数启动、同步和错误检查串联起来。代码使用 CUDA 运行时 **API**(*CUDA Runtime API*), 并用宏统一检查错误。

```
1  #include <cuda_runtime.h>
2
3  #include <cmath>
4  #include <cstdio>
5  #include <cstdlib>
6  #include <vector>
7
8  #define CUDA_CHECK(call) \
9      do { \
10         cudaError_t err__ = (call); \
11         if (err__ != cudaSuccess) { \
12             std::fprintf(stderr, "CUDA error at %s:%d: %s\n", \
13                 __FILE__, __LINE__, cudaGetErrorString(err__)); \
14             std::exit(EXIT_FAILURE); \
15         } \
16     } while (0)
17
18 __global__ void vecAddKernel(const float* A_d,
19                             const float* B_d,
20                             float* C_d,
21                             int n) {
22     const int i = blockIdx.x * blockDim.x + threadIdx.x;
23     if (i < n) {
24         C_d[i] = A_d[i] + B_d[i];
25     }
26 }
```

```
28 void vecAddGPU(const float* A_h, const float* B_h,
29               float* C_h, int n) {
30     if (n <= 0) {
31         return;
32     }
33
34     const size_t bytes = static_cast<size_t>(n) * sizeof(float);
35     float *A_d = nullptr, *B_d = nullptr, *C_d = nullptr;
36
37     CUDA_CHECK(cudaMalloc(reinterpret_cast<void**>(&A_d), bytes));
38     CUDA_CHECK(cudaMalloc(reinterpret_cast<void**>(&B_d), bytes));
39     CUDA_CHECK(cudaMalloc(reinterpret_cast<void**>(&C_d), bytes));
40
41     CUDA_CHECK(cudaMemcpy(A_d, A_h, bytes, cudaMemcpyHostToDevice));
42     CUDA_CHECK(cudaMemcpy(B_d, B_h, bytes, cudaMemcpyHostToDevice));
43
44     constexpr int threadsPerBlock = 256;
45     const int numBlocks =
46         (n + threadsPerBlock - 1) / threadsPerBlock;
47
48     vecAddKernel<<<numBlocks, threadsPerBlock>>>(A_d, B_d, C_d, n);
49     CUDA_CHECK(cudaGetLastError());
50     CUDA_CHECK(cudaDeviceSynchronize());
51
52     CUDA_CHECK(cudaMemcpy(C_h, C_d, bytes, cudaMemcpyDeviceToHost));
53
54     CUDA_CHECK(cudaFree(A_d));
55     CUDA_CHECK(cudaFree(B_d));
56     CUDA_CHECK(cudaFree(C_d));
57 }
```

```
59 int main() {
60     constexpr int n = 1 << 20;
61     std::vector<float> A(n), B(n), C(n);
62
63     for (int i = 0; i < n; ++i) {
64         A[i] = static_cast<float>(i);
65         B[i] = 2.0f * static_cast<float>(i);
66     }
67
68     vecAddGPU(A.data(), B.data(), C.data(), n);
69
70     for (int i = 0; i < n; ++i) {
71         const float expected = A[i] + B[i];
72         if (std::fabs(C[i] - expected) > 1.0e-5f) {
73             std::fprintf(stderr, "Mismatch at %d\n", i);
74             return EXIT_FAILURE;
75         }
76     }
77
78     std::printf("Vector addition passed for n=%d.\n", n);
79     return EXIT_SUCCESS;
80 }
```

9.1 程序结构

1. `main` 在主机端创建并初始化三个向量；
2. `vecAddGPU` 为三个向量分配设备内存；
3. 两次主机到设备复制把输入送入 GPU；
4. 启动配置根据 n 自动计算线程块数；
5. 每个设备线程计算一个 $C[i]$ ，越界线程被条件屏蔽；
6. 主机等待核函数完成并检查错误；
7. 设备到主机复制取回结果，随后释放设备资源；
8. `main` 在 CPU 上验证每一个结果元素。

正确性与性能是两个层次

本程序首先保证结果正确、资源被释放、错误能被报告。它仍会为每次函数调用重新分配内存并往返复制数据；对于包含多个 GPU 阶段的真实程序，通常应让数据在设备端保留更久，减少重复分配与传输。

10 例题与完整推导

10.1 例题一：读取核函数启动配置

给定

```
kernel<<<VECTOR_N, ELEMENT_N>>>(d_C, d_A, d_B, ELEMENT_N);
```

三重尖括号中的第一个参数是线程块数，第二个参数是每块线程数。因此

$$\text{每块线程数} = \text{ELEMENT_N}$$

且

$$\text{网格总线程数} = \text{VECTOR_N} \times \text{ELEMENT_N}.$$

变量名不会改变语法位置的含义；即使命名不理想，也必须按 CUDA 启动规则解释。

10.2 例题二：每线程处理 8 个元素

已知向量长度 $n = 16000$ ，每个线程处理 $k = 8$ 个输出元素，线程块大小 $T = 256$ 。

第一步，求逻辑上需要的工作线程数：

$$L = \left\lceil \frac{n}{k} \right\rceil = \frac{16000}{8} = 2000.$$

第二步，求线程块数：

$$B = \left\lceil \frac{L}{T} \right\rceil = \left\lceil \frac{2000}{256} \right\rceil = 8.$$

第三步，求实际启动线程数：

$$N_{\text{launch}} = BT = 8 \times 256 = 2048.$$

因此答案为

$$2048 \text{ 个线程}.$$

其中 48 个线程没有完整工作。实现线程粗化 (*thread coarsening*) 时，线程 q 可令起始元素为 kq ，并对其负责的每个元素分别检查是否小于 n 。

10.3 例题三：局部变量有多少份

核函数启动 $B = 512$ 个线程块，每块 $T = 256$ 个线程。总线程数为

$$BT = 512 \times 256 = 131072.$$

若核函数中声明一个普通自动局部变量，则每个线程拥有自己的逻辑副本，因此整个执行生命周期中共有

131072 份

线程私有版本。这里计算的是逻辑变量实例数；编译器可能把变量放入寄存器，也可能因资源压力使用局部内存，但线程私有语义不变。

10.4 特殊长度输入

$$B = \left\lceil \frac{4097}{256} \right\rceil = 17, \quad N_{\text{launch}} = 17 \times 256 = 4352, \quad E = 4352 - 4097 = 255.$$

该例达到单个线程块大小下“额外线程数”的最大可能值 $T - 1$ ，说明边界检查不是罕见特例，而是通用模板的一部分。

11 常见错误、检查清单与公式表

11.1 常见错误

错误	后果与修正
把 $\lll B, T \ggg$ 的顺序记反	第一个参数是线程块数，第二个参数是每块线程数。
写成 $\text{ceil}(n/T)$ ，但 n, T 都是整数	整数除法先截断， ceil 已无作用。改用 $(n+T-1)/T$ 或浮点除法后取整。
遗漏 $\text{if } (i < n)$	最后一个线程块中的额外线程可能越界访问。
在主机端解引用设备指针	主机普通代码不应直接访问设备地址；通过 CUDA API 或核函数使用。

错误	后果与修正
不检查 CUDA 返回码	内存不足、非法配置或设备执行错误会被延迟发现，调试困难。
直接用 CPU 计时包围核函数启动	异步启动导致测到的主要是提交开销。计时结束前必须同步，后续宜使用 CUDA 事件。
假设线程块按编号顺序执行	CUDA 可按任意顺序调度线程块；不同线程块应在没有全局同步的情况下保持独立。
忘记释放设备内存	长时间运行或共享 GPU 环境中会造成资源泄漏与后续分配失败。

11.2 写一维 CUDA 核函数前的检查清单

1. 明确一个线程负责的输出元素或数据片段；
2. 写出全局索引公式 $i = bT + t$ ；
3. 计算 $B = \lceil n/T \rceil$ ；
4. 对不整除情况加入边界检查；
5. 明确每个指针位于主机还是设备；
6. 检查分配、复制、启动、执行与释放的错误状态；
7. 不对不同线程块的执行先后作假设；
8. 验证结果后再讨论性能。

11.3 核心公式表

问题	公式
一维全局线程索引	$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$
需要的线程块数	$B = \lceil n/T \rceil = \lfloor (n + T - 1)/T \rfloor$
实际启动线程数	$N_{\text{launch}} = BT$
额外线程数	$E = BT - n$, 且 $0 \leq E < T$
k 元素/线程的逻辑线程数	$L = \lceil n/k \rceil$
k 元素/线程的线程块数	$B = \lceil L/T \rceil$
n 个 <code>float</code> 的字节数	$S = n \cdot \text{sizeof}(\text{float})$
局部变量逻辑副本数	网格总线程数 BT

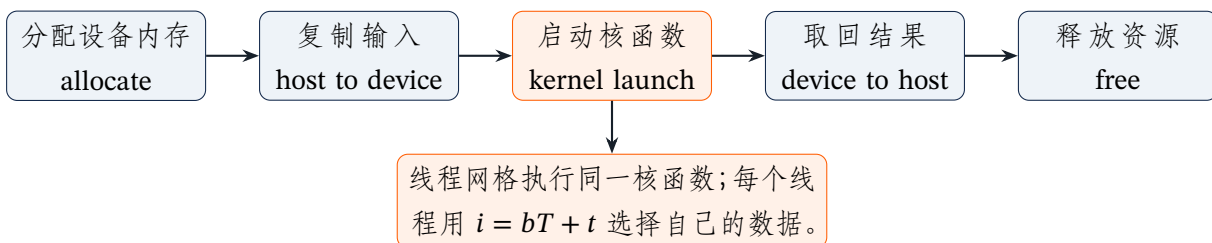
11.4 术语速查

中文术语	英文标准名称
数据并行	data parallelism
任务并行	task parallelism
线程 / 线程块 / 网格	thread / thread block / grid
核函数 / 核函数启动	kernel / kernel launch
单程序多数据	Single Program Multiple Data (SPMD)
主机 / 设备	host / device
主机内存 / 设备全局内存	host memory / device global memory
边界检查	boundary check
向上取整除法	ceiling division
异步调用 / 同步点	asynchronous call / synchronization point
函数限定符	function qualifier
即时编译	just-in-time compilation (JIT compilation)

12 综合框架与关键结论

数据并行 CUDA 程序的基本结构可以归纳为以下完整链条：

1. 从数据依赖出发，定义每个线程的独立工作；
2. 用主机代码管理数据与调度，用设备核函数执行高并行计算；
3. 用网格-线程块-线程层次和 $i = bT + t$ 建立执行到数据的映射；
4. 用显式内存管理完成分配、传输、计算、回传与释放；
5. 用向上取整启动和边界检查覆盖任意长度输入；
6. 用 NVCC 编译主机与设备代码，并正确处理异步执行与错误状态；
7. 先验证正确性，再进行性能优化。



关键公式

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}, \quad B = \frac{n + T - 1}{T},$$

if ($i < n$) 保护边界, $\langle\langle B, T \rangle\rangle$ 依次表示线程块数与每块线程数.